

[← Go Back](#)

Hardware

03. Uploading Sketches to the M4 Core on Arduino Portenta X8

04. Data Exchange Between Python® on Linux & Arduino Sketch

05. Managing Containers with Docker on Portenta X8

06. Using FoundriesFactory® Waves Fleet Management

07. Deploy a Custom Container with Portenta X8 Manager

08. How To Build a Custom Image for Your Portenta X8

09. How To Update Your Portenta X8

10. Data Logging with MQTT, Node-RED, InfluxDB and Grafana

11. Output WebGL Content on a Screen

12. Multi-Protocol Gateway With Portenta X8 & Max Carrier

13. Running WordPress & Database Containers on Portenta X8

14. Portenta X8 Firmware Release Notes

15. Edge AI Flow Monitoring on Portenta X8 with Docker

16. Getting Started with ROS 2 and Turtlesim on Portenta X8

Home / Hardware / Portenta X8 / **15. Edge AI Flow Monitoring on Portenta X8 with Docker**

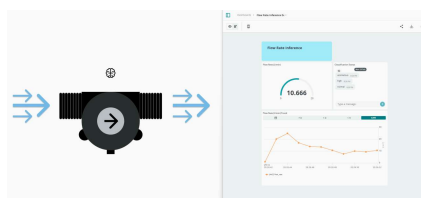
15. Edge AI Flow Monitoring on Portenta X8 with Docker

Learn how to create and deploy an Edge Impulse model on Portenta X8 using Docker containers and a flow sensor for real time anomaly detection.

Author · Taddy Ho Chung · Last revision · 03/04/2025

Overview

Flow sensors provide valuable insights for fluid-based industrial systems by monitoring circulation with observable points of interest and detecting anomalies such as leaks, blockages or pressure variations. Traditional methods often rely on threshold-based detection, which may miss subtle deviations indicating potential failures. Integrating machine learning with Edge Impulse enables real-time pattern classification, improving anomaly detection and predictive maintenance processes.



This application note shows how to implement a flow anomaly detection

ON THIS PAGE

Overview

Goals

Hardware and Software Requirements +

Machine Learning Model for Flow Anomaly Detection +

Anomaly Detection System Setup +

Flow Anomaly Detector Model Development +

System Architecture and Data Flow +

Running Flow Rate Inference with Docker +

Additional Resources

Support +

Help

deployed Edge Impulse model. The Portenta X8 is ideal for this task, offering sensor data acquisition, real-time processing and on-device AI inference in a Linux-based environment using Remote Procedure Call (RPC) for data exchange.

The system captures flow rate data, processes it with a trained model and classifies flow conditions. Results can be used to develop alerts, system responses or cloud updates via Arduino Cloud for remote monitoring and analysis. This approach can help optimize and develop fluid-based industrial systems by combining local AI processing with cloud connectivity and analytics.

Goals

The application note has the following objectives:

- Monitor fluid movement using a flow sensor.

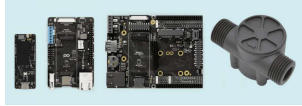
- Classify flow patterns in real time with a machine learning model trained in Edge Impulse.

- Deploy the trained model inside a Docker container on the Portenta X8.

- Share all collected data and classification results to Arduino Cloud for remote monitoring.

Hardware and Software

Requirements



Required hardware

Hardware Requirements

This application note uses the Portenta X8, integrating a flow sensor for real time fluid monitoring. The required hardware includes:

Portenta X8 (x1)

Portenta Carrier Family: **Hat Carrier / Mid Carrier** (x1)

Flow Sensor (e.g. YFS201) (x1)

USB-C® cable (x1)

Wi-Fi® Access Point or Ethernet with Internet access (x1)

Software Requirements

To start with this application, make sure to have your Portenta X8 running on the latest Linux image.

Ensure your Portenta X8 has the latest Linux image. Check [this section of Portenta X8's user manual](#) to verify that your Portenta X8 is up-to-date.

For the smooth functioning of the Portenta Hat Carrier with the Portenta X8, it is crucial to have at least Linux **image version > 746** on the Portenta



X8. To update your board to the latest image, use the [Portenta X8's Arduino Wizard Experience](#) method or [manually flash it](#), downloading the most recent

To develop and deploy the flow monitoring system, the following software tools and platforms are required:

Edge Impulse for model training and deployment

[Edge Impulse® CLI](#), to manage Machine Learning models, we can install it following these [instructions](#).

Docker for running inference models on Portenta X8

[Arduino IDE 2.0+](#) or [Arduino Web Editor](#)

The [Arduino Create Agent](#)

The [Arduino Cloud](#). If you do not have an account, you can create one for free inside [cloud.arduino.cc](#).

[Edge Impulse project example](#). This is a ready-to-use public Edge Impulse project for this application. You can clone it to explore the dataset, model architecture and training pipeline.

Download the Project Code

The complete project files can be downloaded here:

[Portenta X8 onboard inference](#)

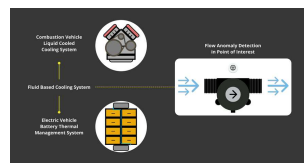
[Portenta X8 onboard inference with Arduino Cloud](#)

Machine Learning

Anomaly Detection

Machine learning enables industrial systems to analyze sensor data, recognize patterns and make data-driven decisions. This application implements a machine learning model to classify flow patterns in real-time, detecting anomalies that may indicate leaks, blockages or irregular pressure fluctuations.

By leveraging Edge Impulse, the model is trained to distinguish between normal and abnormal flow conditions based on present sensor data, allowing for predictive maintenance and operational efficiency.



Flow anomaly detection
with ML generalized
application example

Flow anomaly detection is particularly useful in systems that rely on consistent mass airflow or liquid circulation to maintain optimal performance. For example, most vehicle powerplants require stable coolant delivery to regulate engine temperature.

At the same time, industrial and medical cooling systems must maintain actual flow rates to prevent overheating or component degradation. The system can detect early warning signs of potential failures by classifying flow conditions, allowing proactive

Applications of Flow Rate Categorization

This machine learning model can be applied in different industrial and automotive domains where monitoring and controlling fluid flow at different points are important, based on the key application areas where it involves small systems:

Application Area	Examples
Small Automotive Cooling Systems	- Engine cooling for motorcycles, ATVs, and small passenger vehicles (<1.5L displacement). - Electric vehicle (EV) battery thermal management to prevent overheating. - Turbo intercooler water circuits for efficient heat dissipation.
Industrial Equipment & Electronics Cooling	- CNC spindle cooling systems ensuring precision machining. - Laser cooling loops for maintaining stable laser operation. - Small heat exchanger loops used in industrial automation and process control.
Medical & Laboratory Cooling	- Medical imaging devices (MRI, CT scanners) requiring controlled cooling

Application Area	Examples
	used in chemical processing and biomedical research.

Flow Rate Categorization and Anomaly Detection

The trained model categorizes flow conditions based on real-time measurements from the sensor. The following classification table defines normal and anomalous flow conditions for this application note:

Flow Rate (L/min)	Category	Potential Issues & Notes
<5 L/min	Anomalous (Too Low)	Possible blockage, pump failure, cavitation or air pockets in the system.
5 L/min	Low Flow	Typical for idle operation or minimal cooling demand.
10 L/min	Normal Flow	Steady-state operation under standard cooling requirements.
15+ L/min	High Flow	Increased demand. This may indicate an oversized pump or excessive

Flow Rate (L/min)	Category	Potential Issues & Notes
>20-30 L/min	Anomalous (Too High)	Risk of turbulence, leaks, pressure spikes and reduced heat exchanger efficiency.

By classifying flow rates into these categories, the system can detect abnormal conditions early and take corrective action before performance issues or failures occur.

This approach provides reliable operation in corresponding automotive, industrial and medical cooling applications while reducing maintenance costs and unplanned downtime.

Anomaly Detection System Setup

This application does not require complex wiring for operation. However, proper hardware setup is required for reliable data acquisition, model inference and cloud integration. The following components are required for system deployment:

- Portenta X8:** The main processing unit, running Edge Impulse inference inside a Docker container.
- Portenta Hat/Mid Carrier:** Provides easy access to

Flow Sensor (e.g. YFS201):

Measures real-time flow rate and detects anomalies.

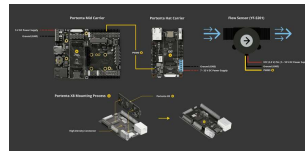
USB-C Cable: Connects the Portenta X8 to a development PC for setup and debugging.

Wi-Fi/Ethernet Access:

Enables cloud connectivity for data visualization in Arduino Cloud.

Hardware Wiring Overview

The system wiring follows a straightforward configuration:



System wiring & power options

Portenta X8: mounted on the **Portenta Mid/Hat Carrier**, provides access to pinout interfaces and **Wi-Fi** or **Ethernet** connection management for cloud communication.

Flow sensor: connected to the corresponding pins on the **Portenta Mid/Hat Carrier**.

USB-C cable: used for initial setup, debugging and Edge Impulse data collection.

Powering Options

Both the **Portenta Mid Carrier** and **Portenta Hat Carrier** support multiple power input options, depending on the deployment requirements:

Carrier Board	USB-C Power (5V)	External DC Power
Portenta Mid Carrier	For development, debugging and short-term testing	5 VDC
Portenta Hat Carrier	For development, debugging and short-term testing	7 - 32 VDC

Setting up the system correctly ensures smooth data collection, machine learning inference, and cloud-based monitoring. With the hardware configured, the next steps focus on data acquisition, model training and deployment.

Flow Anomaly Detector Model Development

Model Design

The machine learning model is trained to recognize different flow conditions based on real-time sensor data. A dataset of flow sensor readings is collected at a predefined frequency, capturing variations in fluid movement under different operating conditions.

The raw sensor data is preprocessed using digital signal processing (DSP) techniques to extract relevant features, which are

of classifying normal and anomalous flow states.

Data Acquisition

Sensor data needs to be collected across different flow conditions to build a reliable machine learning model. This ensures that the model can accurately distinguish between normal and abnormal flow states based on continuous measurements.

The flow sensor connected to the Portenta X8 captures flow rate readings and sends the data to Edge Impulse using the Edge Impulse Data Forwarder. This process allows active monitoring and labeling of flow patterns under different operating conditions.

To collect training data, the Portenta X8 (or compatible Portenta family) continuously measures the real-time flow rate (L/min) from the sensor. It streams it to Edge Impulse via the Data Forwarder.

The M4 Core captures sensor readings at a 1Hz sampling rate and sends them over serial output.

These values are labeled as

Normal, Low, High or

Anomalous, providing a balanced

dataset for training and validation.

Once enough labeled data is collected, it is split into training and testing datasets. The model is then trained on different flow conditions while maintaining a separate set for validation.

Capturing Flow Sensor Data on

The Portenta X8's M4 microcontroller reads flow sensor data and sends it via serial output in CSV format, making it compatible with Edge Impulse for model training.

The following Arduino sketch (`sensor-data-generation.ino`) configures the Portenta X8 to read and send flow rate values at one second intervals.

```
1  #include <Arduino.h>
2  #include <FlowSensor.h>
3
4  #if defined(ARDUINO_PORTENTA_X8)
5  #define SerialDebug Serial
6  #else
7  #define SerialDebug Serial
8  #endif
9
10 // Define Flow Sensor 1
11 #define SENSOR_TYPE YF-S601
12 #define SENSOR_PIN PC_1
13
14 FlowSensor flowSensor(SENSOR_PIN, SENSOR_TYPE);
15
16 // Define 1Hz Sampling
17 #define INTERVAL_MS 1000
18
19 static unsigned long lastTime = 0;
20
21 // Interrupt function 1
22 void count() {
23     flowSensor.count();
24 }
25
26 void setup() {
27     SerialDebug.begin(115200);
28     while (!SerialDebug) {}
29 }
```

Streaming Flow Data to Edge Impulse

Once the Portenta X8 is connected via USB and the Arduino sketch is

rate readings into the Edge Impulse platform.

Before running the data forwarder, make sure that Edge Impulse CLI is installed on your system. If not, install it using:

```
1 npm install -g edge-impul
```

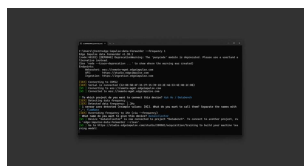
 For detailed Edge Impulse CLI installation instructions, please refer to the [official Edge Impulse documentation](#).

Run the following command to start streaming sensor data from the Portenta X8:

```
1 edge-impulse-data-forward
```

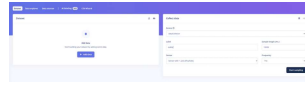
The `--frequency 1` flag sets the data forwarder to  send one sample per second, matching the 1Hz sampling rate of the Arduino sketch.

When prompted, select the Edge Impulse project to which you want to send data.

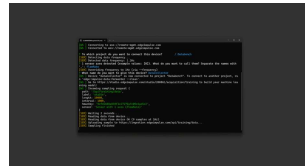


Data forwarder
initialization

prompts for naming the data channels.

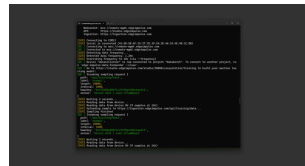


Data forwarder
connected with Edge
Impulse



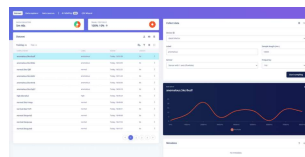
Data acquisition with
Edge Impulse

Once configured, Edge Impulse begins receiving real time flow rate data.



Continuous data
acquisition with Edge
Impulse

The collected flow rate samples are labeled and stored in the Edge Impulse dataset.



Initial Edge Impulse
dataset



For more detailed information
on data forwarding, please
refer to [Edge Impulse's official](#)

After data collection, the labeled dataset is used for feature extraction and model training, so that the system can classify flow anomalies in real-time.

Feature Extraction

Raw sensor data alone is not sufficient for effective anomaly detection. Digital Signal Processing (DSP) techniques are applied to extract meaningful features from the collected flow data to improve model accuracy. These techniques improve distinguishing between normal and anomalous flow conditions by transforming raw time series data into structured feature sets.

Time-Series Analysis:

Captures how the flow rate changes over time, helping identify gradual trends and fluctuations. This is essential for detecting slowly developing anomalies, such as pipe clogging or pump wear.

Spectral Analysis: Converts time-domain signals into the frequency domain, highlighting patterns that may not be evident in the raw data. This helps identify vibrational inconsistencies, leaks or sudden pressure spikes.

Statistical Features:

Computes key statistical values to represent the data in Mean, Variance and Standard deviation. Each representation captures the average flow rate, measures the degree of fluctuation and identifies how much the values deviate from the norm, respectively.

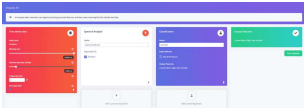
These extracted features form the input dataset used to train the

The *Impulse Design* in Edge Impulse structures the feature extraction pipeline, ensuring data is properly preprocessed before being passed to the learning model.

Time-Series Data Block:

Parameter	Value
Window size	10,000 ms (10 seconds)
Window increase	1,000 ms (1 second)
Frequency	1 Hz
Zero-padding	Enabled (Ensures consistency in data length)

This configuration allows flow rate sampling at one second intervals, providing a detailed view of flow dynamics. The following image shows how the flow rate input is structured before processing:



Impulse Creation

Train/Test Split Process

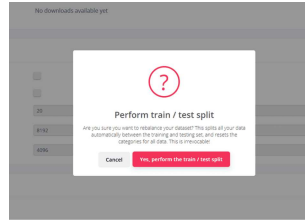
After feature extraction, the dataset needs to be split into *Training* and *Testing* sets to validate the model's performance.

Train/Test Split Process: The dataset is automatically divided into:

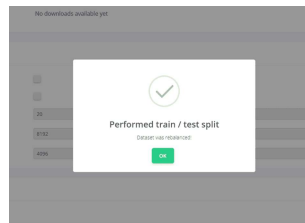
- Training set (~80%)
- Testing set (~20%)

It makes that the model generalizes

is done to maintain category consistency.



Dataset Split Warning



Dataset Split Confirm

The first image shows the warning before dataset splitting, while the second confirms the successful rebalance of the dataset.

Once split, the dataset contains labeled flow rate data across different operational conditions.

Each data sample is labeled as:

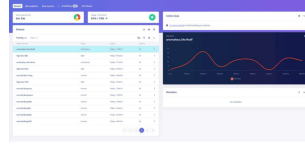
Normal: Flow rate within expected range.

Low: Below normal levels (may indicate blockages or pump inefficiency).

High: Above normal levels, (may indicate pressure spikes or leaks).

Anomalous: Unusual variations that require further investigation.

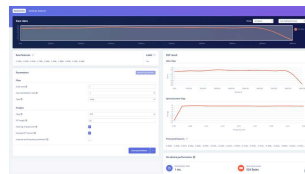
The following image shows the final dataset distribution with balanced class representation:



Dataset Split Confirm

Spectral Analysis: Frequency Domain Interpretation

Now, the image below shows the DSP feature extraction process, where the raw flow rate data is converted into its spectral representation:

DSP feature extraction
process

Raw Data Processing: The top left section displays the original time series flow rate signal. **Spectral**

Power Analysis: The lower right section shows the frequency-domain representation, highlighting dominant spectral components.

Feature Extraction Parameters:

The left panel includes key configuration options such as:

FFT length: Determines the frequency resolution of the spectral transformation.

Log of spectrum: Enhances feature scaling by applying logarithmic compression.

Overlapping FFT frames: Reduces information loss between samples.

The generated spectral features are then used to train the machine

components correlate to physical flow behavior:

Low-frequency components typically correspond to gradual pressure variations in the system, which may indicate normal operation or slow leaks.

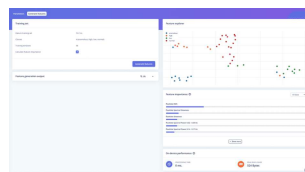
Mid-frequency components often relate to flow instability, which can signal developing faults such as pipe constrictions.

High-frequency oscillations may suggest rapid flow disturbances caused by sudden leaks, valve malfunctions or pump cavitation.

By converting raw data into its spectral form, the model can detect subtle fluctuations that may not be obvious in time-series analysis.

Feature Importance Analysis and Model Optimization

Once the spectral features are extracted, an analysis is performed to determine which features contribute most to distinguishing between normal and anomalous flow conditions. The following image shows the feature explorer and importance ranking:



DSP feature explorer and importance characterization

Feature Explorer: Displays

Feature importance and correlation

different flow states (Normal, Low, High, Anomalous).

Feature Importance

Ranking: Lists the most critical spectral features, with

FlowRate RMS and

FlowRate Spectral Skewness

being the top indicators of anomalies.

The model can classify different flow conditions by analyzing these distributions based on extracted DSP features. The FlowRate RMS and Spectral Skewness are key indicators of anomalies for following reasons:

FlowRate RMS (Root Mean

Square): Represents the overall magnitude of flow fluctuations. A significant increase in RMS values may indicate irregular flow behavior, sudden bursts or system instabilities.

FlowRate Spectral Skewness:

Measures the asymmetry of the spectral distribution. A shift in skewness can highlight unexpected fluctuations in flow rate, often correlated with anomalies such as pressure surges, valve defects or sudden leaks.

By prioritizing the most important features, the model can be updated to be more efficient and accurate in detecting deviations, reducing the risk of false positives and ensuring robust real time anomaly classification.

With feature extraction completed, the system is now ready for model training. The extracted features will be used to train a classification model that distinguishes between different flow states

Please refer to the [Edge Impulse Sensor Fusion](#)

 [Documentation](#) for further insights into DSP based sensor fusion and anomaly detection.

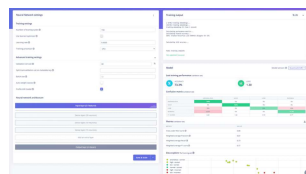
Model Training

Using Edge Impulse, the processed data is used to train a machine learning model capable of classifying flow conditions. The objective of the model is to:

- Recognize normal flow patterns based on historical data.

- Detect anomalous flow conditions, such as unexpected fluctuations, obstructions or irregular flow rates.

Different machine learning techniques, such as decision trees, neural networks or anomaly detection models, may be considered to determine the most effective approach. This project uses a neural network classifier to map extracted spectral features to flow states. The following image shows the *Neural Network settings and training process*:



Neural Network settings and training process

Neural Network Architecture is configured with:

Input: 34 features

Hidden layers: 30 neurons
(Dense Layer) / 10 neurons
(Dense Layer) / 15 neurons
(Dense Layer)

Output layer: 4 classes
(Normal, Low, High,
Anomalous)

The network uses *ReLU activation functions* in hidden layers and *Softmax activation* in the output layer to predict the most probable class.

The Neural Network Training Configuration is as follows:

Parameter	Value
Training cycles	150
Learning rate	0.0005
Processor	CPU based training
Validation set size	30% of the dataset
Batch size	32 (not enabled in this run)

The model's *quantized (int8) version* is used for deployment to optimize inference speed and reduce memory usage.

Performance Metrics

Once training is complete, the model's accuracy is evaluated using a confusion matrix and performance scores. The key results include:

Metric	Value
Overall Model Accuracy	73.3%
Weighted Average Precision	0.87
Weighted Average Recall	0.73



For a detailed explanation of each metric, please refer to the [evaluation metrics documentation from Edge Impulse](#).

The confusion matrix provides insights into model performance across different categories:

Anomalous flow conditions are 100% correctly classified.

High flow rates are also detected with 100% accuracy.

Low flow rates have 60% accuracy, with some misclassification into the **Normal** category.

Normal flow conditions have 71.4% accuracy, but 28.6% are incorrectly classified as **low** flow.

Interpreting Training Results

The trained model is capable of detecting high flow and anomalous conditions, performing accurate recognition in these categories. Additionally, the quantized model keeps a balance between accuracy and computational efficiency, making it appropriate for embedded deployment on the Portenta X8. The model is capable of reducing false positives with a precision score of 0.87, allowing reliable anomaly detection.

However, low flow conditions present the most significant challenge, with the highest misclassification rate. The model occasionally misclassifies low flow as normal, indicating difficulty distinguishing borderline cases. To

cycles, adjustments in feature selection and further data augmentation may be required to improve recognition. Integrating feature scaling techniques could also help refine classification accuracy, particularly for subtle variations in flow conditions.

Here, it is possible to look into the following details:

Hyperparameter tuning:

Adjust the learning rate and number of neurons to improve classification.

Data balancing: Ensure equal representation of all flow conditions to reduce bias.

Refining the model can improve classification accuracy and improve real-time flow anomaly detection.

Deployment and Onboard Inference

Once trained, the machine learning model will be used to deploy to the Portenta X8 inside a Docker container, allowing real-time inference directly at the edge. Instead of pulling the container directly, we will use the container image, arguments and ports provided by Edge Impulse to build a custom Docker container for the Portenta X8.

This allows the model to run within our system configuration while keeping compatibility with the hardware and software constraints.

As new sensor data is collected, the model classifies flow conditions, providing feedback to the system. If

can trigger predefined actions, such as:

- Activating an alarm to alert operators of potential system failures.

- Sending notifications to Arduino Cloud for remote monitoring and logging.

- Adjusting system parameters dynamically to prevent failures or improve efficiency.

By processing data locally on the Portenta X8, this approach reduces dependence on external servers, reducing response time and providing continuous operation in network-limited environments.

Preparing the Docker Deployment for Portenta X8

Instead of pulling the container directly, Edge Impulse provides the necessary components to build a custom Docker container for Portenta X8. These include:

- Container Image URL:** The base inference container.

- Arguments:** API key for authentication and runtime configuration.

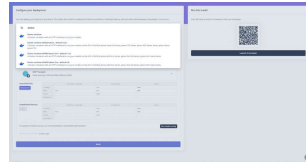
- Ports to Expose:** Defines how external applications can access the inference server.

This setup allows for custom integration with the Portenta X8's existing Docker-based environment, providing compatibility with other applications and services running on the device.

Selecting Docker Deployment

The deployment process starts by

target environment within Edge
Impulse:

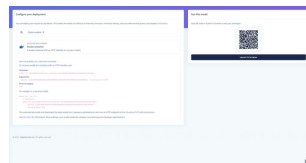


Docker container as
deployment method

Edge Impulse provides multiple options, including Jetson based deployments and general-purpose Docker containers. **The standard Docker container with an HTTP interface is used for this application.**

Building a Custom Docker Container for Portenta X8

For deployment on the Portenta X8, we use the provided container image, arguments and ports to build a custom container that integrates with the system's existing services.



Docker container
deployment parameters

The inference workflow consists of the following steps:

Sensor Data Acquisition: The flow rate sensor measures present values.

Feature Extraction: DSP and spectral analysis are applied to structure data.

Model Prediction: The trained model classifies the current

Response Actions: If an anomaly is detected, the system takes corresponding action.

By running inference directly on the Portenta X8, this approach can provide latency reduction and autonomous anomaly detection. With the Docker container ready, the next step is to compose and integrate the necessary components for deployment on the Portenta X8.

For additional details on Docker based inference, refer to the [official Edge Impulse Docker Deployment documentation](#).

System Architecture and Data Flow

The flow anomaly detection system has real-time sensor data acquisition, machine learning inference and cloud-based monitoring. The structure consists of three main layers:

Data Collection Layer: The Portenta X8 reads flow sensor data, capturing variations in fluid movement.

Processing & Inference Layer: The Edge Impulse trained model runs inside a Docker container on the Portenta X8, classifying flow patterns and detecting anomalies.

Cloud & Visualization Layer: The system forwards classification results to Arduino Cloud, allowing remote monitoring with visualized information.

Data Flow Overview

The flow sensor will measure fluid movement continuously and send data to the Portenta X8's M4 core. The M4 core processes this data and makes it accessible to the Linux layer using **Remote Procedure Call (RPC)**.



For more details about Remote Procedure Call (RPC) and its implementation, refer to the [dedicated tutorial](#).

A Python script on the Linux side receives sensor data from the M4 core. It forwards it to the Edge Impulse inference model, which runs inside a Docker container. The model classifies the flow condition as **Normal, Low, High or Anomalous**.

The system logs an alert and records classification results if an anomaly is detected. These results are stored locally for quick decision making and sent to Arduino Cloud for remote monitoring, historical analysis and anomaly tracking.

This integration optimizes performance by balancing edge processing with cloud integration:

Onboard Inference: The Portenta X8 processes sensor data locally, working on low-latency anomaly detection while reducing bandwidth usage.

Cloud Based Monitoring: Arduino Cloud stores classification results, enabling long-term data tracking, predictive maintenance and real-time alerts if configured.

The system performs on demand anomaly detection by combining active edge inference with cloud analytics while providing historical insights for predictive maintenance and system optimization.

Running Flow Rate Inference with Docker

We need to have the [hardware setup as explained here](#) before proceeding.

The M4 core needs to have the `rpc-flow-sensor.ino` uploaded to collect and send sensor data via RPC.

```
1 #include <Arduino.h>
2 #include <RPC.h>
3 #include <SerialRPC.h>
4 #include <FlowSensor.h>
5
6 // Define Serial Debug
7 #if defined(ARDUINO_PO)
8 #define SerialDebug Ser
9 #else
10 #define SerialDebug Ser
11 #endif
12
13 // Define Flow Sensor 1
14 #define SENSOR_TYPE YFS
15 #define SENSOR_PIN PC_
16
17 // Create Flow Sensor (
18 FlowSensor flowSensor($
19
20 void count() {
21     flowSensor.count();
22 }
23
24 // Function to get Flow
25 float getFlowRate() {
26     flowSensor.read();
27     float flowRate = flow
28
```

The inference engine runs inside a Docker container, providing compatibility with the system.

The *Docker Compose* configuration defines the inference container and sensor data collection service.

```
1 networks:
2   sensorfusion:
3
4 services:
5   cad:
6     image: arduino/pyth
7     build: .
8     restart: unless-sto
9     depends_on:
10      - inference
11     tty: true
12     volumes:
13      - '/run/arduino_t
14      - '/sys/devices:/
15      - '/sys/class/pwm
16      - '/sys/bus/iio:/
17      - '/var/sota:/var
18     extra_hosts:
19      - "m4proxy:host-gat
20     devices:
21      - '/dev/gpiochip6
22      - '/dev/gpiochip1
23      - '/dev/gpiochip2
24      - '/dev/gpiochip3
25      - '/dev/gpiochip4
26      - '/dev/gpiochip5
27     networks:
28       sensorfusion:
```

The compose file will require the **container, arguments and port values** that were generated when deployed the machine learning model as explained [here](#).

The `main.py` script receives flow sensor data from the M4 core and sends it to the inference container.

```
1 #!/usr/bin/env python3
2 import os
3 import time
4 import requests # type
5 import signal
6 import sys
7 import json
8 import argparse
9
10 from msgpackrpc import
11
12 m4_proxy_host = 'm4-pr
13 m4_proxy_port = 5001
14 m4_proxy_address = Rpc
15 model_type = os.getenv
16
17 def get_sensors_data_f
18     """Get data from the
19     data = {}
20     sensors = ('flow_rat
21
22     try:
23         client = RpcClie
24         data = {sensor:
25
26     except RpcError.Time
27         print("Unable to
28
```

The complete project files can be downloaded in [this section](#).

Once the docker files are ready, please upload the required files using ADB:

```
1 adb push .\python-ei-sens
```

Running the Inference Model on Docker

With all files in place, navigate to the project directory and build the container:

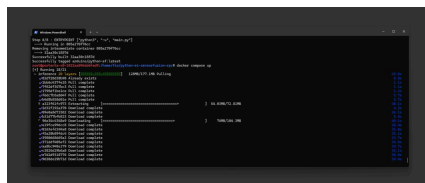

```
1 docker build -t arduino/p
```

Start the application:

```
1 docker compose up
```

You can add `-d` prefix in the end of the command to start the container in the background, allowing you to continue using the terminal without keeping the container logs displayed. If you skip `-d`, the logs from the containers will be shown in the terminal and it will remain attached to the process until you manually stop it.

When the application starts for the first time, the shell shows similar output as follows:



To verify the inference engine is running, check logs:

```
1 docker compose logs -f -n
```

The system will start to collect flow sensor data, process it with the Edge Impulse model inside a Docker container and detect anomalies.

```
1 docker compose up -d
```

```
1 docker exec -it <container>
```

```
1 python3 -u /app/python/main.py
```

Arduino Cloud Integration

Now that the Portenta X8 is running the Docker container with the trained inference model, the next step is integrating Arduino Cloud for active monitoring and logging of anomaly detection results.



To register and configure the Portenta X8 with Arduino Cloud, please refer to [this user manual section](#).

The `main.py` script from [previous section](#) is updated to forward flow rate classification results to Arduino Cloud. Before proceeding, an Arduino Cloud Thing must be set up with a flow rate variable to obtain the Thing ID and Variable ID.

```

1  #!/usr/bin/env python3
2  import os
3  import time
4  import requests
5  import signal
6  import sys
7  import json
8  import argparse
9  from msgpackrpc import
10
11  # Environment variables
12  CLIENT_ID = os.getenv(
13  CLIENT_SECRET = os.get
14  THING_ID = os.getenv('
15  CLASSIFICATION_VARIABLE
16  CONFIDENCE_VARIABLE_ID
17  LOOP_INTERVAL = int(os
18
19  # Arduino IoT Cloud API
20  TOKEN_URL = "https://e
21  PROPERTY_UPDATE_URL =
22
23  # M4 Proxy Configurati
24  m4_proxy_host = "m4-pr
25  m4_proxy_port = 5001
26  m4_proxy_address = Rpc
27
28  # Inference Server Con

```

The script gets API credentials and device identifiers from a `.env` file. Create this file in the project directory with the following content:

```

1  CLIENT_ID=cloud_api_key
2  CLIENT_SECRET=cloud_api_s
3  THING_ID=cloud_thing_id
4  SPACE_ID=cloud_space_id
5  FLOWRATE_VARIABLE_ID=clou
6  CLASSIFICATION_VARIABLE_I

```

The `docker-compose` file is updated to include Arduino Cloud credentials:

```
1 networks:
2   sensorfusion:
3
4 services:
5   cad:
6     image: arduino/pyth
7     build: .
8     restart: unless-sto
9     depends_on:
10      - inference
11     tty: true
12     PYTHONUNBUFFERED:
13     LOOP_INTERVAL: 1
14     CLIENT_ID: ${CLIE
15     CLIENT_SECRET: $+
16     THING_ID: ${THINC
17     FLOWRATE_VARIABLE
18     CLASSIFICATION_VA
19   volumes:
20     - '/run/arduino_t
21     - '/sys/devices:/
22     - '/sys/class/pwm
23     - '/sys/bus/iio:/
24     - '/var/sota:/var
25   extra_hosts:
26     - "m4proxy:host-gat
27   devices:
28     - '/dev/gpiochip6
```

Once the files are updated, it needs to be pushed to the Portenta X8 again, navigate to the project directory and use the following commands:

```
1 docker build -t arduino/p
```

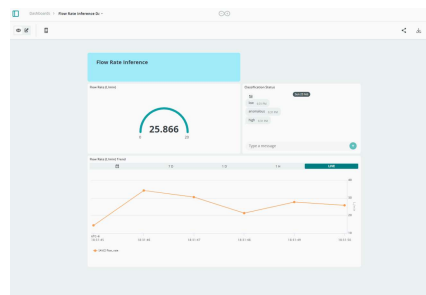
```
1 docker compose --env-file
```

You can add `-d` prefix in the end of the command to start the container in the background, allowing you to continue using the terminal without keeping the container logs displayed. If you skip `-d`, the logs from the containers will be shown in the terminal and it will remain attached to the process until you manually stop it.

The following command can be used to verify the inference engine is running:

```
1 docker compose logs -f -n
```

Once deployed, the system will begin to get flow sensor data, classify it using the trained model and send the results to Arduino Cloud for monitoring and anomaly tracking.



For more controlled execution:

```
1 docker compose --env-file
```

```
1 docker exec -it <container
```

```
1 python3 -u /app/python/ma
```

This integration allows real-time anomaly detection and cloud-based monitoring, combining edge inference on Portenta X8 with Arduino Cloud analytics. Users can remotely track flow rate anomalies, set up alerts and analyze historical trends to improve predictive maintenance of the system's point of interest.

Additional Resources

[Portenta X8 Documentation](#)

[Edge Impulse Documentation](#)

GitHub Repository: [portenta-x8/webinars](#)

Support

If you encounter any issues or have questions while working with the Portenta X8, we provide various support resources to help you find answers and solutions.

Help Center

Explore our [Help Center](#), which

X8. The Arduino Help Center is designed to provide in-depth technical assistance and help you make the most of your device.

[Portenta Family help center page](#)

Forum

Join our community forum to connect with other Portenta X8 users, share your experiences, and ask questions. The forum is an excellent place to learn from others, discuss issues, and discover new ideas and projects related to the Portenta X8.

[Portenta X8 category in the Arduino Forum](#)

Contact Us

Please get in touch with our support team if you need personalized assistance or have questions not covered by the help and support resources described before. We're happy to help you with any issues or inquiries about the Portenta X8.

[Contact us page](#)

Suggest changes

The content on [docs.arduino.cc](#) is facilitated through a public [GitHub repository](#).

Need support?

[Help Center](#)
[Ask the Arduino Forum](#)
[Discover Arduino](#)
[Discord](#)

License

The Arduino documentation is licensed under the [Creative Commons Attribution-Share Alike 4.0](#) license.

wrong, you
can edit
this page
[here](#).

Was this article helpful?

